

PA1 Security on Release of Unverified Plaintext in Encrypt-then-MAC AE Schemes

Bart Mennink and Suprita Talnikar[†]

^{*}Digital Security, Radboud University, Toernooiveld 200-208, Nijmegen,
6525EC, The Netherlands.

^{*}Corresponding author(s). E-mail(s): b.mennink@cs.ru.nl;
suprita.talnikar@ru.nl;

[†]These authors contributed equally to this work.

Abstract

At ASIACRYPT 2014, Andreeva et al. put forward a definition for security of authenticated encryption under release of unverified plaintext. They introduced two notions of plaintext awareness (PA1 and its stronger sibling PA2), suggested to be used in conjunction with confidentiality in case of release of unverified plaintext, as well as the notion of integrity under release of unverified plaintext (INT-RUP). Various efforts have been made to develop a unified model (e.g., Ashur et al., CRYPTO 2017, Chang et al., ToSC 2019(4)). With respect to the analysis of existing and new modes under release of unverified plaintext, most research however has focused on INT-RUP security only. Plaintext awareness is less studied and understood.

In this work, we take a detailed look at the original definitions of PA1 and PA2 security. We observe that the definitions leave too much room for interpretation, and claimed results such as PA1 security of Encrypt-then-MAC are unjustified. The core of the issue lies in the fact that PA1 security is necessarily tied to the implementation of the scheme. To resolve this, we present refined definitions of PA1 and PA2 security. We argue that even for these refined definitions, there is no implementation of Encrypt-and-MAC that is PA1 (nor PA2) secure. For MAC-then-Encrypt, results depend on the actual scheme, as we demonstrate using a negative result and a positive result (from literature, on Romulus-M). Furthermore, we formally prove for Encrypt-then-MAC that (i) there exist implementations that are PA1 insecure and (ii) there exist implementations that are PA1 secure. In other words, Encrypt-then-MAC is insecure under the old definition but secure under the new definition, provided a proper implementation is

used. We apply this observation to Isap v2, finalist in the NIST Lightweight Cryptography competition, where we additionally deal with the complication that the same key is used for encryption and authentication.

Keywords: Authenticated Encryption, Release of Unverified Plaintext, Plaintext Awareness, Isap v2.

1 Introduction

Secure communication is pivotal for the assurance we have in today’s digital environment. Most importantly, we expect that data is sent confidentially, namely that no outsider can read/interpret data, and that it comes with some authentication mechanism, such that no outsider can manipulate the data. The former can be achieved by encryption schemes and the latter by MAC (message authentication code) functions, and both have received a vast amount of research and attention. A mechanism that achieves both goals at the same time is authenticated encryption (AE). Typically, an authenticated encryption scheme Π is considered to consist of a pair of algorithms $\{\mathcal{E}, \mathcal{D}\}$: encryption $\mathcal{E}$ takes as input a key K , a nonce N , associated data A , and message M , and it outputs a ciphertext C encrypting the message and a tag T authenticating the associated data and the message; the decryption $\mathcal{D}$ takes as input a key K , a nonce N , associated data A , ciphertext C , and tag T , and it outputs the message M corresponding to the ciphertext — *but only if* the tag verifies correctly.

The formal introduction of authenticated encryption with security modeling dates back to 2000 [1, 2], and while certain very well-established schemes such as OCB [3–5] and GCM [6] date back to the aughts, authenticated encryption has been chiefly on the rise in the last decade only, mostly thanks to the CAESAR competition [7] and the recently concluded NIST Lightweight Cryptography competition [8].

However, with the rise in popularity of authenticated encryption, possible weaknesses in use-cases appeared. Well-known problems are the issue of nonce-reuse (refer to [9]) and the risk of side-channels (refer to [10, 11]). A weakness in which we are interested is that of instances that release plaintext before the tag is verified. This event may happen in use-cases where there is insufficient secure memory to store the message, real-time problems caused by inefficiency in releasing the data, or even mistakes in implementation. In 2014, Andreeva et al. [12] formalized the idea of security of authenticated encryption in case of release of unverified plaintext (RUP).

The first step in formalizing RUP security is to separate the decryption algorithm $\mathcal{D}$ into “plain” decryption $\mathcal{D}$ and verification $\mathcal{V}$.¹ This split is easily made for typical authenticated encryption schemes, possibly including some duplicate code in the algorithms of $\mathcal{D}$ and $\mathcal{V}$. Then, Andreeva et al. consider authentication and encryption separately. For authentication, they formalize the notion of INT-RUP (for integrity under release of unverified plaintext), which reasonably closely resembles the notion of integrity for the conventional formalism of authenticated encryption (see Definition 4).

¹Admittedly, using the same parameter $\mathcal{D}$ seems annoying at first, but it turns out to cause no confusion later in the paper.

For confidentiality, they resort to the concept of plaintext awareness in conjunction to confidentiality, first introduced in the context of public-key encryption by Bellare and Rogaway [13] and later defined without random oracles by Bellare and Palacio [14]. In a nutshell, the idea of plaintext awareness is that, if an attacker has access to the encryption and decryption functionality, the decryption functionality reveals no surprising data to the attacker and can as well be replaced by an artificial keyless functionality called an “extractor”. The notion comes in two flavors, namely plaintext awareness 1 and 2 (see Definitions 5 and 6), which differ in how this extractor operates and in the power the adversary has.

Since the formalism of Andreeva et al., there has been a lot of work on analyzing (solely) integrity under release of unverified plaintext [15–19]. There have also been two works that introduced unified RUP models, of Ashur et al. [20] and Chang et al. [21], and applied them to a few modes. Beyond that, the RUP notions related to (solely) confidentiality, namely plaintext awareness 1 and 2, have rarely been considered in isolation. The main exceptions to this are certain results from Andreeva et al. on (adjusted) McOE-G [22], APE [23], and certain general constructions, and a plaintext awareness proof of the mode of Romulus-M [24, 25].

1.1 Related Authenticated Encryption Definitions

The work of Andreeva et al. [12] on security in case of release of unverified plaintext is embedded in a large body of works on the development of models for authenticated encryption with added security properties. Predating RUP security is the work of Boldyreva et al. [26], that considered security of authenticated encryption with decryption leakages, where decryption could result in certain errors and thus leak information. The work of Andreeva et al. itself, as mentioned above, separated RUP security into integrity under release of unverified plaintext and plaintext awareness. The two above-mentioned follow-up works of Ashur et al. [20] and Chang et al. [21] presented RUPAE and AERUP, respectively, with an aim to unify RUP security into one definition.

From a broader perspective, Hoang et al. [27] introduced robust authenticated encryption (RAE), a single security definition which encompasses various considerations, including the requirement that a decryption algorithm should release only harmless information when presented with an invalid ciphertext and a repeated nonce. Barwell et al. [28] introduces subtle authenticated encryption (SAE), that is meant to be a general notion capturing ideas of Boldyreva et al.’s model, the RUP security model, as well as the RAE security model. Compared to the definitional framework of Andreeva et al. on RUP security, SAE considers a three-oracle form and makes a few modifications to the decryption and verification oracles.

Here, it is worth noting that these overarching notions of RAE and SAE, and to a lesser extent RUPAE and AERUP, simply require a stronger authenticated encryption scheme. Stated differently, authenticated encryption schemes failing to meet “just” plaintext awareness or “just” INT-RUP security typically also fail to meet these more general security notions. In fact, as we shall see next, describing schemes that achieve plaintext awareness (and in consequence, more generalized notions) is not easy unless a more extensive scheme is taken that directly meets the stronger notions.

The authors of [29] compare their notion of channel simulatability (CS) with PA1 and PA2. For example, they claim that PA1 roughly corresponds to a notion of decryption simulatability where the simulator has unrestricted access to the transcript. However, they do note that their formulation would not suffice to guarantee chosen-ciphertext security.

1.2 Our Contributions

We take a closer look at the definition of plaintext awareness of Andreeva et al. [12] and observe that, in fact, the current definition is counter-intuitive and too much open for interpretation. As a good example, consider an Encrypt-then-MAC authenticated encryption mode, where a message is first encrypted and the ciphertext is subsequently authenticated to obtain the tag. Intuitively, this mode should achieve plaintext awareness: as the authentication is performed over the ciphertext, one can verify the tag based on the ciphertext simply before encrypting, and *only start decrypting if the tag is correctly verified!* Note that this is not necessarily in contradiction with the idea of releasing unverified plaintext: a typical use-case of releasing unverified plaintext, as explained above, is when there is insufficient secure memory to store the message before being released, and for the Encrypt-then-MAC example this is not an issue. On the other hand, despite the intuition that Encrypt-then-MAC should achieve plaintext awareness, it does not, as one can describe a trivial attack that succeeds with probability close to 1.

A closer look at the situation reveals that it all boils down to the exact configuration of the authenticated encryption scheme into the triplet of functions $\{\mathcal{E}, \mathcal{D}, \mathcal{V}\}$. Stated differently, it is not appropriate to call an authenticated encryption scheme plaintext aware, but rather the implementation or configuration of that authenticated encryption scheme. The limitation is described in more detail in Section 3.1.

Based on this observation, we introduce refined security notions for plaintext awareness. This formalism, however, turns out to be much more subtle than it seems. The reason is that for virtually any authenticated encryption scheme Π , one can come up with a *cheating* variant $\{\mathcal{E}, \mathcal{D}, \mathcal{V}\}$ that achieves plaintext awareness in some way, where, for example, intermediate computations that go against the idea of security against release of unverified plaintext are made within $\mathcal{D}$. To avoid such cheating approaches, we first define (in Section 3.2) the notation of a *valid configuration*. Intuitively, a configuration is valid if $\mathcal{D}$ does not perform intermediate elementary computations that output plaintext data. Then, we say that an authenticated encryption scheme is plaintext aware *for a certain implementation* if its corresponding configuration is plaintext aware.

We next apply the new notions and definitions to the conventional authenticated encryption modes Encrypt-and-MAC, MAC-then-Encrypt, and Encrypt-then-MAC [2]. This application yields the following observations:

- (i) there exists no configuration of Encrypt-and-MAC that is plaintext aware;
- (ii) no exclusive result can be claimed for MAC-then-Encrypt;
- (iii) the default configuration of Encrypt-then-MAC is not plaintext aware, but a more logical configuration of Encrypt-then-MAC is plaintext aware.

For result (i), we describe a constant-time attack that succeeds with overwhelming probability. For result (ii), we demonstrate that the result carries over for some instantiations, but observe additionally that an SIV [30] style approach makes the mode plaintext aware (see also the proof of Romulus-M [24, 25]). For result (iii), we formally prove that this configuration achieves plaintext awareness as long as the underlying MAC function is unforgeable. These results are formally derived in Section 4.

1.3 Outline

We first give some preliminaries in Section 2, including the definitions of authenticated encryption and release of unverified plaintext as taken from Andreeva et al. [12]. Then, in Section 3, we take a closer look at the definitions of plaintext awareness, we describe limitations, the definition of a valid configuration, and the refined security models. We apply the models to the three conventional authenticated encryption modes in Section 4. We conclude the work in Section 5.

2 Preliminaries

Let $m, n \in \mathbb{N}$ such that $m \leq n$. By $\{0, 1\}^n$ we denote the set of n -bit strings, and by $\{0, 1\}^*$ the set of arbitrary-length strings. For $X \in \{0, 1\}^n$, $\text{left}_m(X)$ outputs the m leftmost bits, and $\text{right}_m(X)$ the m rightmost bits. For $A \in \{0, \dots, 2^n - 1\}$, $\langle A \rangle_n$ denotes the encoding of A as an n -bit string. If $\mathcal{S}$ is a finite set, we denote by $S \xleftarrow{\$} \mathcal{S}$ the uniform random selection of an element S from $\mathcal{S}$.

An adversary is an algorithm A that is given access to one or more oracles, denoted A^f , and either outputs a bit or wins if a certain event occurs. For any two oracles f and g , the distinguishing advantage of A is defined as

$$\Delta_A(f; g) := |\Pr[A^f \rightarrow 1] - \Pr[A^g \rightarrow 1]|.$$

Here, the probabilities are taken over the random coins of A , f , and g .

Throughout, we use $k, m, t \in \mathbb{N}$.

2.1 Pseudorandom Function

We will need the concept of a pseudorandom function. A pseudorandom function F is a function that takes as input a key from $\{0, 1\}^k$, a message from $\{0, 1\}^*$, and outputs a tag in $\{0, 1\}^*$ of arbitrary length (this length may be fixed by the application, or vary and be fixed using an additional input to the function). The *PRF* advantage of F against an adversary A is defined as

$$\text{PRF}_F(A) := \Delta_A(F_K; \text{RO}).$$

Here, the probabilities are taken over the random key K , the random oracle RO , and the random coins of A .

2.2 Authenticated Encryption

An authenticated encryption scheme Π consists of a pair of algorithms $\mathcal{E}$ and $\mathcal{D}$. Here,

$$\mathcal{E} : \{0, 1\}^k \times \{0, 1\}^m \times \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^* \times \{0, 1\}^t$$

is the encryption and authentication algorithm, that takes as input a key from $\{0, 1\}^k$, nonce (or IV) from $\{0, 1\}^m$, associated data from $\{0, 1\}^*$, and message from $\{0, 1\}^*$, and outputs a ciphertext in $\{0, 1\}^*$ and tag in $\{0, 1\}^t$. The decryption function

$$\mathcal{D} : \{0, 1\}^k \times \{0, 1\}^m \times \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^t \rightarrow \{0, 1\}^* \cup \{\perp\}$$

on the other hand takes as input a key from $\{0, 1\}^k$, nonce or IV from $\{0, 1\}^m$, associated data from $\{0, 1\}^*$, ciphertext from $\{0, 1\}^*$, and tag from $\{0, 1\}^t$, and outputs either a message in $\{0, 1\}^*$ or the dedicated $\perp$ -sign if authentication fails. The two algorithms satisfy the correctness condition, in that for any $(K, N, A, M) \in \{0, 1\}^k \times \{0, 1\}^m \times \{0, 1\}^* \times \{0, 1\}^*$,

$$\mathcal{D}(K, N, A, \mathcal{E}(K, N, A, M)) = M.$$

We assume $\mathcal{E}$ and $\mathcal{D}$ to be length preserving, i.e., for a message M of size $|M|$ bits, the corresponding ciphertext will also be $|M|$ bits. For particular authenticated encryption schemes, certain conditions on the nonce or IV may apply, such as uniqueness upon encryption.

We will conventionally abbreviate $\mathcal{E}(K, N, A, \cdot)$ to $\mathcal{E}_K^{N, A}$ and $\mathcal{D}(K, N, A, \cdot, \cdot)$ to $\mathcal{D}_K^{N, A}$. In certain cases, the key space $\{0, 1\}^k$ consists of a Cartesian product of two sets, and we abuse notation to abbreviate $\mathcal{E}((K, L), N, A, \cdot)$ to $\mathcal{E}_{K, L}^{N, A}$ (and likewise for $\mathcal{D}$).

We state the main security notions of confidentiality and integrity, below.

2.2.1 Confidentiality

Confidentiality assures that different encryptions appear indistinguishable. We can describe it as a distinguishing game where the adversary has access to either $\mathcal{E}_K$ or a function $\$$ that generates random strings of appropriate length on each input.

Definition 1 (Confidentiality). *Consider an authenticated encryption scheme Π . Let $\$$ be a function that always responds with a uniform random ciphertext and tag of the exact same length as one would expect from $\mathcal{E}_K$. The CONF advantage of Π against an adversary A is defined as*

$$\text{CONF}_{\Pi}(A) := \Delta_A(\mathcal{E}_K; \$).$$

Here, the probabilities are taken over the random key K , the random function $\$$, and the random coins of A . Adversary A may be restricted by some conditions on the N 's it queries. This restriction depends on the specific use-case.

2.2.2 Integrity

Integrity on the other hand, captures the case that an attacker cannot forge a tag. We can describe it in an indistinguishability setting where the attacker has access to the encryption functionality $\mathcal{E}_K$ and either a decryption functionality $\mathcal{D}_K$ or a dedicated function that always outputs $\perp$. For convenience, however, we adopt the security game where the attacker “wins” if $\mathcal{D}_K$ ever succeeds.

Definition 2 (Integrity). *Consider an authenticated encryption scheme Π . The INT advantage of Π against an adversary $\mathbf{A}$ is defined as*

$$\text{INT}_{\Pi}(\mathbf{A}) := \Pr [\mathbf{A}^{\mathcal{E}_K, \mathcal{D}_K} \text{ forges}] .$$

Here, the probability is taken over the random key K and the random coins of $\mathbf{A}$. By “forges”, we mean that $\mathcal{D}$ returns a valid message (i.e., not the $\perp$ sign). Adversary $\mathbf{A}$ is not allowed to query the outputs of its first oracle to its second oracle.

2.3 Conventional Modes of Operation

In 2000, Bellare and Namprempre [2] formalized the three most logical generic composition methods for authenticated encryption based on an encryption scheme and a MAC scheme. To discuss these, we first have to describe the interface of these modular building blocks.

Let $k_{\text{enc}} \in \mathbb{N}$. An encryption scheme consists of an encryption function enc and a decryption function dec with the following interfaces:²

$$\begin{aligned} \text{enc} &: \{0, 1\}^{k_{\text{enc}}} \times \{0, 1\}^m \times \{0, 1\}^* \rightarrow \{0, 1\}^* , \\ \text{dec} &: \{0, 1\}^{k_{\text{enc}}} \times \{0, 1\}^m \times \{0, 1\}^* \rightarrow \{0, 1\}^* . \end{aligned}$$

We assume enc and dec to be length preserving. We abbreviate $\text{enc}(K, N, \cdot)$ to enc_K^N and $\text{dec}(K, N, \cdot)$ to dec_K^N . The two algorithms satisfy the correctness condition, in that for any $(K, N, M) \in \{0, 1\}^{k_{\text{enc}}} \times \{0, 1\}^m \times \{0, 1\}^*$,

$$\text{dec}(K, N, \text{enc}(K, N, M)) = M .$$

Let $k_{\text{mac}} \in \mathbb{N}$. A MAC scheme consists of an authentication function mac and a verification function vfy with the following interface:

$$\begin{aligned} \text{mac} &: \{0, 1\}^{k_{\text{mac}}} \times \{0, 1\}^m \times \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^t , \\ \text{vfy} &: \{0, 1\}^{k_{\text{mac}}} \times \{0, 1\}^m \times \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^t \rightarrow \{\top, \perp\} . \end{aligned}$$

We abbreviate $\text{mac}(K, N, A, \cdot)$ to $\text{mac}_K^{N, A}$ and $\text{vfy}(K, N, A, \cdot)$ to $\text{vfy}_K^{N, A}$. The two algorithms satisfy the correctness condition, in that for any $(K, N, A, M) \in \{0, 1\}^{k_{\text{mac}}} \times$

²There could be slight variations in different types of compositions, depending on presence and size of the domains.

$$\{0, 1\}^m \times \{0, 1\}^* \times \{0, 1\}^*,$$

$$\text{vfy}(K, N, A, M, \text{mac}(K, N, A, M)) = \top.$$

Looking ahead, we will rely on the unforgeability of a MAC function, which is in fact almost identical to Definition 2 except for the name of the game and some cosmetic changes. We state it for completeness.

Definition 3 (Unforgeability). *Consider an authentication function mac and its verification function vfy . The UNF advantage of mac against an adversary A is defined as*

$$\text{UNF}_{\text{mac}}(A) := \Pr [A^{\text{mac}_K, \text{vfy}_K} \text{ forges}].$$

Here, the probability is taken over the random key K and the random coins of A . By “forges”, we mean that vfy returns $\top$. Adversary A is not allowed to query the outputs of its first oracle to its second oracle.

Using these definitions, we can discuss the three generic composition methods as formalized and analyzed by Bellare and Namprempre [2], below. We note that these composition methods operate on a key space that is the Cartesian product of the two key spaces for encryption and authentication, as in $\{0, 1\}^k = \{0, 1\}^{k_{\text{enc}}} \times \{0, 1\}^{k_{\text{mac}}}$, or $k = k_{\text{enc}} + k_{\text{mac}}$.

2.3.1 Encrypt-and-MAC

Encrypt-and-MAC basically encrypts and authenticates data in parallel and concatenates the inputs. Formally, the authenticated encryption scheme $\Pi = \{\mathcal{E}, \mathcal{D}\}$ is defined as follows:

$$\begin{aligned} \mathcal{E}_{K,L}^{N,A} : M &\mapsto (C, T) = (\text{enc}_K^N(M), \text{mac}_L^{N,A}(M)), \\ \mathcal{D}_{K,L}^{N,A} : (C, T) &\mapsto \begin{cases} M = \text{dec}_K^N(C) & \text{if } \text{vfy}_L^{N,A}(\text{dec}_K^N(C), T) = \top, \\ \perp & \text{if } \text{vfy}_L^{N,A}(\text{dec}_K^N(C), T) = \perp. \end{cases} \end{aligned}$$

Encrypt-and-MAC is generically considered insecure as contrived instantiations may result in an insecure authenticated encryption scheme.

2.3.2 MAC-then-Encrypt

MAC-then-Encrypt first computes the authentication tag over the entire input, and then enters this piece of data into the encryption algorithm. This can be done either as separate associated data, or as appendix to the plaintext (the formalism that we opt for below). Formally, the authenticated encryption scheme $\Pi = \{\mathcal{E}, \mathcal{D}\}$ is defined as follows:

$$\mathcal{E}_{K,L}^{N,A} : M \mapsto (C, T) = (\text{enc}_K^N(M \parallel \text{mac}_L^{N,A}(M)), \text{mac}_L^{N,A}(M)),$$

$$\mathcal{D}_{K,L}^{N,A} : (C, T) \mapsto \begin{cases} M = \text{left}_{|C|-|T|}(\text{dec}_K^N(C)) & \text{if } \text{vfy}_L^{N,A}(\text{left}_{|C|-|T|}(\text{dec}_K^N(C)), T) = \top, \\ \perp & \text{if } \text{vfy}_L^{N,A}(\text{left}_{|C|-|T|}(\text{dec}_K^N(C)), T) = \perp. \end{cases}$$

Here, we note that typically $\mathcal{E}_{K,L}^{N,A}$ does not output T along with C as it is implicit in C ; here, however, we do so to make our notations more compact. We stress that this does not limit our observations in any way. MAC-then-Encrypt exposes risks of padding oracle attacks as authentication is performed over the message. Looking ahead to Section 4.2, it appears that for certain instantiations of MAC-then-Encrypt, the slightly weaker security guarantees offered by this mode could make this problematic; for others, such as Romulus-M [25], this is less the case.

2.3.3 Encrypt-then-MAC

Encrypt-then-MAC first encrypts the message and then authenticates the ciphertext instead of the message. Formally, the authenticated encryption scheme $\Pi = \{\mathcal{E}, \mathcal{D}\}$ is defined as follows:

$$\begin{aligned} \mathcal{E}_{K,L}^{N,A} : M &\mapsto (C, T) = (\text{enc}_K^N(M), \text{mac}_L^{N,A}(\text{enc}_K^N(M))), \\ \mathcal{D}_{K,L}^{N,A} : (C, T) &\mapsto \begin{cases} M = \text{dec}_K^N(C) & \text{if } \text{vfy}_L^{N,A}(C, T) = \top, \\ \perp & \text{if } \text{vfy}_L^{N,A}(C, T) = \perp. \end{cases} \end{aligned}$$

Encrypt-then-MAC is considered the most secure variant, in particular as it guarantees ciphertext integrity. Example authenticated encryption schemes that in some way follow the Encrypt-then-MAC design are GCM [6] and EAX [31]. Later on in Section 4.3, we will consider a more modern authenticated encryption scheme Isap v2 [32] in detail. APE [23] can also be considered to be Encrypt-then-MAC, although the mapping is a bit more subtle as decryption is different.

2.4 Release of Unverified Plaintext

Releasing unverified plaintext captures the case where the decryption functionality $\mathcal{D}$ accidentally or due to implementation releases plaintext before the tag is verified. To formalize release of unverified plaintext, Andreeva et al. [12] proposed to describe an authenticated encryption scheme Π as a triplet of algorithms instead, namely an encryption algorithm $\mathcal{E}$, a decryption functionality $\mathcal{D}$, and a verification functionality $\mathcal{V}$:

$$\begin{aligned} \mathcal{E} : \{0, 1\}^k \times \{0, 1\}^m \times \{0, 1\}^* \times \{0, 1\}^* &\rightarrow \{0, 1\}^* \times \{0, 1\}^t, \\ \mathcal{D} : \{0, 1\}^k \times \{0, 1\}^m \times \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^t &\rightarrow \{0, 1\}^*, \\ \mathcal{V} : \{0, 1\}^k \times \{0, 1\}^m \times \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^t &\rightarrow \{\top, \perp\}. \end{aligned}$$

The encryption algorithm typically remains unchanged, and the “old” decryption functionality can be considered to now be split into a decryption and verification

algorithm. We require a similar correctness condition as before, namely that for any $(K, N, A, M) \in \{0, 1\}^k \times \{0, 1\}^m \times \{0, 1\}^* \times \{0, 1\}^*$,

$$\begin{aligned}\mathcal{D}(K, N, A, \mathcal{E}(K, N, A, M)) &= M, \\ \mathcal{V}(K, N, A, \mathcal{E}(K, N, A, M)) &= \top.\end{aligned}$$

We recall the main security notions for authenticated encryption under release of unverified plaintext from Andreeva et al. [12], below.

2.4.1 Integrity Under Release of Unverified Plaintext

We start with integrity under release of unverified plaintext, which is merely included for completeness.

Definition 4 (Integrity Under Release of Unverified Plaintext). *Consider an authenticated encryption scheme Π . The INT-RUP advantage of Π against an adversary $\mathbf{A}$ is defined as*

$$\text{INT-RUP}_{\Pi}(\mathbf{A}) := \Pr [\mathbf{A}^{\mathcal{E}_K, \mathcal{D}_K, \mathcal{V}_K} \text{ forges}] .$$

Here, the probability is taken over the random key K and the random coins of $\mathbf{A}$. By “forges”, we mean that $\mathcal{V}$ returns $\top$. Adversary $\mathbf{A}$ is not allowed to query the outputs of its first oracle to its third oracle.

2.4.2 Plaintext Awareness

More relevant to us are the notions of plaintext awareness. Plaintext awareness intuitively covers the idea that, for an adversary having access to the decryption functionality $\mathcal{D}$, there is no valid way to retrieve anything it could have known beforehand. This is formalized using an *plaintext extractor*: we consider a security game where the adversary has access to encryption and either decryption or the extractor, and it has to determine which world it is communicating with.

Plaintext awareness comes in two flavors. The first one is PA1 security. This notion involves an efficient plaintext extractor that mimics the decryption oracle *using the query history of the encryption oracle*.

Definition 5 (Plaintext Awareness 1). *Consider an authenticated encryption scheme Π . Let $\mathbf{E}$ be a PA1 extractor. The PA1 advantage of Π against an adversary $\mathbf{A}$, relative to $\mathbf{E}$, is defined as*

$$\text{PA1}_{\Pi}^{\mathbf{E}}(\mathbf{A}) := \Delta_{\mathbf{A}}(\mathcal{E}_K, \mathcal{D}_K; \mathcal{E}_K, \mathbf{E}) .$$

Here, the probabilities are taken over the random key K and the random coins of $\mathbf{A}$ and $\mathbf{E}$. The extractor $\mathbf{E}$ has access to the query history of $\mathcal{E}_K$.

The second flavor is PA2 security. The definition differs only in the fact that now the extractor has no knowledge of the query history of the encryption oracle, but the adversary is not allowed to relay an encryption response to the decryption oracle (to avoid trivial attacks).

Definition 6 (Plaintext Awareness 2). *Consider an authenticated encryption scheme Π . Let E be a PA2 extractor. The PA2 advantage of Π against an adversary A , relative to E , is defined as*

$$\text{PA2}_{\Pi}^E(A) := \Delta_A(\mathcal{E}_K, \mathcal{D}_K; \mathcal{E}_K, E).$$

Here, the probabilities are taken over the random key K and the random coins of A and E . Adversary A is not allowed to query the outputs of its first oracle to its second oracle.

PA2 security is stronger than PA1 security: Andreeva et al. [12] proved that if a scheme is PA2 secure, then it is PA1 secure.

3 Refining Plaintext Awareness

We first describe the limitation of the current definitions in Section 3.1. We define the refined security models in Section 3.3. These new definitions require a concept of “valid configurations”, which we describe in Section 3.2.

3.1 Limitation

The limitation of the current model becomes quickly visible when considering Encrypt-then-MAC. Note that a plain configuration of Encrypt-then-MAC into encryption, decryption, and verification is the following:

$$\mathcal{E}_{K,L}^{N,A} : M \mapsto (C, T) = (\text{enc}_K^N(M), \text{mac}_L^{N,A}(C)), \quad (1a)$$

$$\mathcal{D}_{K,L}^{N,A} : (C, T) \mapsto M = \text{dec}_K^N(C), \quad (1b)$$

$$\mathcal{V}_{K,L}^{N,A} : (C, T) \mapsto \top/\perp = \text{vfy}_L^{N,A}(C, T). \quad (1c)$$

In other words, encryption is the typical way, decryption simply decrypts the ciphertext, and verification verifies the ciphertext, noting that the tag authenticated the ciphertext.

It is in fact obvious to see that this configuration is *not* PA1 secure. The reason is that any call to $\mathcal{D}_{K,L}^{N,A}$ leaks the message corresponding to the ciphertext, regardless of the tag. A formal argument is included in Section 4.3. Note that this is not only in contradiction to the claim of Andreeva et al. [12] (“For instance, to avoid releasing unverified plaintext into a device with insecure memory [46], the two-pass Encrypt-then-MAC composition can be used: a first pass to verify the MAC, and a second to decrypt the ciphertext.”), it is also against intuition: indeed, one can simply decide not to decrypt anything before the tag is verified!

3.2 Valid Configurations

The core issue of the limitation described in Section 3.1 is that plaintext awareness seems to be dependent on how to actually define the decryption and verification functionalities; for certain authenticated encryption modes, different such configurations are possible.

However, formalizing what configurations are valid or invalid, is rather delicate. Consider for example the case of MAC-then-Encrypt. The plain configuration is the following

$$\mathcal{E}_{K,L}^{N,A} : M \mapsto (C, T) = (\text{enc}_K^N(M \| T), \text{mac}_L^{N,A}(M)), \quad (2a)$$

$$\mathcal{D}_{K,L}^{N,A} : (C, T) \mapsto M = \text{left}_{|C|-|T|}(\text{dec}_K^N(C)), \quad (2b)$$

$$\mathcal{V}_{K,L}^{N,A} : (C, T) \mapsto \top / \perp = \text{vfy}_L^{N,A}(\text{left}_{|C|-|T|}(\text{dec}_K^N(C)), T), \quad (2c)$$

This configuration seems valid (but PA1 insecure, as we will see in Section 4.2). Alternatively, one could define

$$\mathcal{E}_{K,L}^{N,A} : M \mapsto (C, T) = (\text{enc}_K^N(M \| T), \text{mac}_L^{N,A}(M)), \quad (3a)$$

$$\mathcal{D}_{K,L}^{N,A} : (C, T) \mapsto \begin{cases} M = \text{left}_{|C|-|T|}(\text{dec}_K^N(C)) \\ \quad \text{if } \text{vfy}_L^{N,A}(\text{left}_{|C|-|T|}(\text{dec}_K^N(C)), T) = \top, \\ \text{abort} \quad \text{if } \text{vfy}_L^{N,A}(\text{left}_{|C|-|T|}(\text{dec}_K^N(C)), T) = \perp, \end{cases} \quad (3b)$$

$$\mathcal{V}_{K,L}^{N,A} : (C, T) \mapsto \top / \perp = \text{vfy}_L^{N,A}(\text{left}_{|C|-|T|}(\text{dec}_K^N(C)), T), \quad (3c)$$

where **abort** is a dedicated sign stating that decryption will not proceed. Intuitively, this configuration may achieve plaintext awareness as $\mathcal{D}_{K,L}^{N,A}$ never outputs the $\text{left}_{|C|-|T|}(\text{dec}_K^N(C))$ unless verification is correct, but one can consider this configuration cheating as it also already computes $\text{left}_{|C|-|T|}(\text{dec}_K^N(C))$ as intermediate step in (3b)! This is against the original spirit of security under release of unverified plaintext, where any “chunk” of plaintext is released *whenever it is computed*.

Thus, intuitively, we can say that a configuration is *valid* if the decryption function $\mathcal{D}_{K,L}^{N,A}$ never computes plaintext as an intermediate step. However, formalizing this in full generality is rather subtle. Indeed: the above reasoning only holds for the conventional composition methods where the ciphertext is decrypted into the corresponding plaintext *at once*, and it does not readily generalize to cases where **enc** and **dec** are based on underlying smaller building blocks and only part of the plaintext is computed as an intermediate step. In addition, it does not capture silly cases where no pure plaintext data is computed intermediately, but rather checksums, hashes, and so on.

The bottom line is that we can only define what a valid configuration is with respect to the evaluation of underlying modular building blocks. For the conventional authenticated encryption methods, the modular building blocks are **enc**, **dec**, **mac**, and **vfy**, but the granularity can go finer into building blocks being block ciphers, permutations, and so on.

We are now ready to define the notion of a valid configuration.

Definition 7 (Valid Configuration). *Consider a configuration $\{\mathcal{E}, \mathcal{D}, \mathcal{V}\}$ of an authenticated encryption scheme Π . The configuration is a valid configuration if $\mathcal{D}$ does not perform intermediate elementary evaluations that reveal message data. Here, elementary refers to the modular building blocks upon which $\mathcal{D}$ is built, and intermediate refers to the fact that the evaluation outcomes may be discarded/stored before being output.*

We admit that it is tricky to describe modular building blocks in more detail: they can for example be as large as `dec` in black-box generic composition schemes like Encrypt-and-MAC, MAC-then-Encrypt, and Encrypt-then-MAC, but they may as well be as small as an $\oplus$ in case of stream encryption. The intermediateness is added in the definition to avoid cheating examples as above which go against the spirit of “releasing unverified plaintext”: in words, *any* elementary evaluation that outputs message data in a valid configuration actually reveals that data to the attacker. Note that “separation” of the AE decryption algorithm does not mean physically “breaking” it into decryption (ciphertext $\rightarrow$ plaintext) and verification (with output of plaintext/ $\top$ or $\perp$) algorithms. Rather, it is a way of viewing the algorithm so as to look at plaintext awareness in a more structured manner; the algorithm(s) remain the same (“unbroken”), and we even call them functionalities (not algorithms) in our paper (Section 2.4). Hence, other security notions (related to RUP or not) still apply to the AE after our PA security analysis.

3.3 New Definitions

Having defined the notion of a valid configuration in Section 3.2, we are ready to refine the notions of plaintext awareness from Definitions 5 and 6. In fact, the definitions are still rather similar: the refined definitions differ *only in the fact* that it is bound to a particular configuration.

Definition 8 (Refined Plaintext Awareness 1). *Consider a valid configuration of an authenticated encryption scheme Π . Let $\mathbf{E}$ be a PA1 extractor. The refined PA1 advantage of Π against an adversary $\mathbf{A}$, relative to $\mathbf{E}$, is defined as*

$$\text{refPA1}_{\Pi}^{\mathbf{E}}(\mathbf{A}) := \Delta_{\mathbf{A}}(\mathcal{E}_K, \mathcal{D}_K; \mathcal{E}_K, \mathbf{E}) .$$

Here, the probabilities are taken over the random key K and the random coins of $\mathbf{A}$ and $\mathbf{E}$. The extractor $\mathbf{E}$ has access to the query history to $\mathcal{E}_K$.

Definition 9 (Refined Plaintext Awareness 2). *Consider a valid configuration of an authenticated encryption scheme Π . Let $\mathbf{E}$ be a PA2 extractor. The refined PA2 advantage of Π against an adversary $\mathbf{A}$, relative to $\mathbf{E}$, is defined as*

$$\text{refPA2}_{\Pi}^{\mathbf{E}}(\mathbf{A}) := \Delta_{\mathbf{A}}(\mathcal{E}_K, \mathcal{D}_K; \mathcal{E}_K, \mathbf{E}) .$$

Here, the probabilities are taken over the random key K and the random coins of $\mathbf{A}$ and $\mathbf{E}$. Adversary $\mathbf{A}$ is not allowed to query the outputs of its first oracle to its second oracle.

4 Refined Plaintext Awareness of Conventional Authenticated Encryption Modes

We will explore PA1 security of the conventional authenticated encryption modes of Section 2.3, treating Encrypt-and-MAC in Section 4.1, MAC-then-Encrypt in

Section 4.2, and Encrypt-then-MAC in Section 4.3. In the last section, we also apply it to the mode of Isap v2 [32].

4.1 Encrypt-and-MAC

The default configuration for an Encrypt-and-MAC scheme is $\Pi_{\text{def}}^{\text{EaM}} = \{\mathcal{E}, \mathcal{D}, \mathcal{V}\}$, defined as follows:

$$\mathcal{E}_{K,L}^{N,A} : M \mapsto (C, T) = (\text{enc}_K^N(M), \text{mac}_L^{N,A}(M)), \quad (4a)$$

$$\mathcal{D}_{K,L}^{N,A} : (C, T) \mapsto M = \text{dec}_K^N(C), \quad (4b)$$

$$\mathcal{V}_{K,L}^{N,A} : (C, T) \mapsto \top/\perp = \text{vfy}_L^{N,A}(\text{dec}_K^N(C), T). \quad (4c)$$

It can be easily observed that Encrypt-and-MAC with this default configuration does not achieve PA1 security. Consequently, it also does not achieve PA2 security.

Proposition 1. *The default configuration $\Pi_{\text{def}}^{\text{EaM}}$ for Encrypt-and-MAC is PA1 insecure. Stated differently, for any $\alpha \in \mathbb{N}$ there exists an adversary $\mathbf{A}$ making one query to each of its oracles such that for any extractor $\mathbf{E}$,*

$$\text{refPA1}_{\Pi_{\text{def}}^{\text{EaM}}}^{\mathbf{E}}(\mathbf{A}) \geq 1 - 1/2^\alpha.$$

Proof. Let $\alpha \in \mathbb{N}$. Let $\mathbf{E}$ be any PA1 extractor relative to the Encrypt-and-MAC configuration $\Pi_{\text{def}}^{\text{EaM}}$ of (4). Consider the following adversary $\mathbf{A}$ interacting with either $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_K, \mathcal{D}_K)$ or $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_K, \mathbf{E})$:

1. Fix arbitrary $N \in \{0, 1\}^m$, $A \in \{0, 1\}^*$, $C \in \{0, 1\}^\alpha$, and $T \in \{0, 1\}^t$.
2. Query $\mathcal{O}_2(N, A, C, T)$ to receive M .
3. Query (N, A, M) to $\mathcal{O}_1$ to receive (C', T') .
4. If $C' = C$, return 1, otherwise, return 0.

By construction, in the real world $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_K, \mathcal{D}_K)$, $\Pr[\mathbf{A}^{\mathcal{E}_K, \mathcal{D}_K} \rightarrow 1] = 1$, since $\mathcal{E}_K^{N,A}(\mathcal{D}_K^{N,A}(C, T)) = (C, T')$ (where T' may differ from T). In the ideal world $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_K, \mathbf{E})$ however, we have $C' = C$ with probability $1/|C| = 1/2^\alpha$, however: $\Pr[\mathbf{A}^{\mathcal{E}_K, \mathbf{E}} \rightarrow 1] = 1/2^\alpha$. Thus, for this adversary,

$$\text{refPA1}_{\Pi}^{\mathbf{E}}(\mathbf{A}) = 1 - 1/2^\alpha. \quad \square$$

$\square$

Admittedly, the attack of Proposition 1 only applies to the default configuration of (4). Nonetheless, one can argue that there is no other meaningful configuration that can do better: any other configuration either falls victim to the same attack or is invalid with respect to Definition 7. This follows clearly from the verification algorithm of the default configuration, i.e., (4c): the algorithm *first* needs to decrypt in order to verify, and any adjustment to the decryption functionality of (4b) that would release plaintext only conditioned on an earlier verification attempt would have to decrypt first, and more importantly, run $\text{dec}_K^N(C)$ as an intermediate step. This

would be in violation with the definition of a valid configuration. We henceforth obtain the following corollary.

Corollary 1. *Any valid configuration for Encrypt-and-MAC is PA1 insecure.*

We remark that this reasoning *only* holds for the generic composition of Encrypt-and-MAC, where the functions `enc`, `dec`, `mac`, and `vfy` are treated as elementary building blocks and no finer level of granularity is considered.

4.2 MAC-then-Encrypt

The default configuration for a MAC-then-Encrypt scheme is $\Pi_{\text{def}}^{\text{MtE}} = \{\mathcal{E}, \mathcal{D}, \mathcal{V}\}$, defined as follows, which is basically a duplication of (2):

$$\mathcal{E}_{K,L}^{N,A} : M \mapsto (C, T) = (\text{enc}_K^N(M \| T), \text{mac}_L^{N,A}(M)), \quad (5a)$$

$$\mathcal{D}_{K,L}^{N,A} : (C, T) \mapsto M = \text{left}_{|C|-|T|}(\text{dec}_K^N(C)), \quad (5b)$$

$$\mathcal{V}_{K,L}^{N,A} : (C, T) \mapsto \top / \perp = \text{vfy}_L^{N,A}(\text{left}_{|C|-|T|}(\text{dec}_K^N(C)), T). \quad (5c)$$

Note that this default configuration is fairly identical to (4), with the difference that encryption `enc` and decryption `dec` take the tag T as input.

It turns out that for this case, plaintext awareness is highly dependent on the constructions used. To demonstrate this, we discuss an insecure and a secure instantiation, below.

4.2.1 Insecure Instantiation

Consider MAC-then-Encrypt but with stream encryption. In other words, `enc` is based on an underlying pseudorandom function $F : \{0, 1\}^k \times \{0, 1\}^m \times \mathbb{N} \rightarrow \{0, 1\}^*$ where the third input determines the requested output length, and which is defined as follows:

$$\text{enc}_K^N(M') = F_K(N, |M'|) \oplus M' \quad (6)$$

(a typical instantiation of F_K could be counter mode).

For this construction, the attack of Proposition 1 carries over almost verbatim, but it differs slightly in that the last t bits of decryption need to be ignored. However, this change is merely administrative.

Proposition 2. *Assume one takes the plain streaming mode of (6) for `enc`. The default configuration $\Pi_{\text{def}}^{\text{MtE}}$ for MAC-then-Encrypt is PA1 insecure. Stated differently, for any $\alpha \in \mathbb{N}$ there exists an adversary $\mathbf{A}$ making one query to each of its oracles such that for any extractor $\mathbf{E}$,*

$$\text{refPA1}_{\Pi_{\text{def}}^{\text{MtE}}}^{\mathbf{E}}(\mathbf{A}) \geq 1 - 1/2^\alpha.$$

In fact, the reasoning stretches to any configuration for the MAC-then-Encrypt with counter mode for `enc`, for the same reason as for Corollary 1.

Corollary 2. *Assume one takes plain counter mode of (6) for `enc`. Any valid configuration for MAC-then-Encrypt is PA1 insecure.*

4.2.2 Secure Instantiation

The above instantiation is insecure as one concatenates M and T and feeds $M\|T$ into encryption. If, instead, one *uses* T as nonce in the encryption function, i.e., using $F : \{0,1\}^k \times \{0,1\}^t \times \mathbb{N} \rightarrow \{0,1\}^*$:

$$\text{enc}_K^N(M', T) = F_K(T, |M'|) \oplus M', \quad (7)$$

the attack does not work. This approach is very similar to the SIV [30] approach, and also similar to the MAC-then-Encrypt construction underlying Romulus-M [25], which Iwata et al. proved plaintext aware in [24]. For the simplified setting of using enc of (7), we derive a positive result below.

Theorem 3. *Assume one takes the advanced streaming mode of (7) for enc . The configuration $\Pi_{\text{def}}^{\text{MtE}}$ for MAC-then-Encrypt is PA1 secure. Stated differently, there exists an extractor $\mathbf{E}$ such that for any adversary $\mathbf{A}$ making q queries to its first oracle and p queries to its second oracle,*

$$\text{refPA1}_{\Pi_{\text{smart}}^{\text{EtM}}}^{\mathbf{E}}(\mathbf{A}) \leq \text{UNF}_{\text{mac}}(\mathbf{B}),$$

for some adversary $\mathbf{B}$ making q queries to its first oracle and p queries to its second oracle.

Proof. Consider a PA1 extractor $\mathbf{E}$ that operates as follows on input of any tuple (N, A, C, T) , where we recall that $\mathbf{E}$ has access to the query history of $\mathcal{E}_{K,L}$:

- If there is an $M \in \{0,1\}^*$ and an earlier evaluation $\mathcal{E}_{K,L}(N, A, M) = (C, T)$, return that M .
- Otherwise, return $M\|T$ for a random string $M \xleftarrow{\$} \{0,1\}^{|C|-|T|}$.

We have to argue that for any adversary $\mathbf{A}$, it is hard to distinguish $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_{K,L}, \mathcal{D}_{K,L})$ from $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_{K,L}, \mathbf{E})$.

Note that, whenever the adversary makes a repeated query to $\mathcal{O}_2$ or relays a response from $\mathcal{O}_1$ to $\mathcal{O}_2$, the oracle response is identical in both worlds (due to step 1 of $\mathbf{E}$ above). Suppose the adversary makes a *fresh* query (N, A, C, T) to $\mathcal{O}_2$. In the ideal world (see step 2 of $\mathbf{E}$ above), $\mathbf{E}$ outputs a random M of appropriate length. In the real world (see (5b)), $\mathcal{D}_{K,L}$ outputs

$$M = \text{left}_{|C|-|T|}(\text{dec}_K^N(C, T)) = \text{left}_{|C|-|T|}(F_K(T, |C|) \oplus C).$$

As T is used to diversify F_K , the output will appear like a random string, *except if* $\text{vfy}_L^{N,A}(M, T) = \top$.

More formally, denote by **BAD** the event that, in the real world $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_{K,L}, \mathcal{D}_{K,L})$, any query (N, A, C, T) that $\mathbf{A}$ makes to $\mathcal{D}_{K,L}$ satisfies $\text{vfy}_L^{N,A}(M, T) = \top$. Then, by the fundamental lemma of game playing [33]:

$$\begin{aligned} \text{refPA1}_{\Pi_{\text{smart}}^{\text{EtM}}}^{\mathbf{E}}(\mathbf{A}) &= |\Pr[\mathbf{A}^{\mathcal{E}_K, \mathcal{D}_K} \rightarrow 1] - \Pr[\mathbf{A}^{\mathcal{E}_K, \mathbf{E}} \rightarrow 1]| \\ &\leq |\Pr[\mathbf{A}^{\mathcal{E}_K, \mathcal{D}_K} \rightarrow 1 \mid \neg \text{BAD}] - \Pr[\mathbf{A}^{\mathcal{E}_K, \mathbf{E}} \rightarrow 1]| \end{aligned}$$

$$\begin{aligned}
& + \Pr[\mathbf{A}^{\mathcal{E}_K, \mathcal{D}_K} \text{ sets BAD}] \\
& \leq \Pr[\mathbf{A}^{\mathcal{E}_K, \mathcal{D}_K} \text{ sets BAD}] \\
& = \text{UNF}_{\text{mac}}(\mathbf{B}),
\end{aligned}$$

for some adversary $\mathbf{B}$ making q queries to its first oracle and p queries to its second oracle. $\square$ $\square$

4.3 Encrypt-then-MAC

We can consider the default configuration of Encrypt-then-MAC, $\Pi_{\text{def}}^{\text{EtM}} = \{\mathcal{E}, \mathcal{D}, \mathcal{V}\}$, which is basically a duplication of (1):

$$\mathcal{E}_{K,L}^{N,A} : M \mapsto (C, T) = (\text{enc}_K^N(M), \text{mac}_L^{N,A}(C)), \quad (8a)$$

$$\mathcal{D}_{K,L}^{N,A} : (C, T) \mapsto M = \text{dec}_K^N(C), \quad (8b)$$

$$\mathcal{V}_{K,L}^{N,A} : (C, T) \mapsto \top / \perp = \text{vfy}_L^{N,A}(C, T). \quad (8c)$$

One can argue that Encrypt-then-MAC with this default configuration does not achieve PA1 security; the proof is almost identical to that of Proposition 1, modulo minor changes, and hence omitted.

Proposition 4. *The default configuration $\Pi_{\text{def}}^{\text{EtM}}$ for Encrypt-then-MAC is PA1 insecure. Stated differently, for any $\alpha \in \mathbb{N}$ there exists an adversary $\mathbf{A}$ making one query to each of its oracles such that for any extractor $\mathbf{E}$,*

$$\text{refPA1}_{\Pi_{\text{def}}^{\text{EtM}}}^{\mathbf{E}}(\mathbf{A}) \geq 1 - 1/2^\alpha.$$

However, it is possible to define a smarter configuration for which Encrypt-then-MAC *does* achieve PA1 security. Define $\Pi_{\text{smart}}^{\text{EtM}} = \{\mathcal{E}, \mathcal{D}, \mathcal{V}\}$ as follows:

$$\mathcal{E}_{K,L}^{N,A} : M \mapsto (C, T) = (\text{enc}_K^N(M), \text{mac}_L^{N,A}(C)), \quad (9a)$$

$$\mathcal{D}_{K,L}^{N,A} : (C, T) \mapsto \begin{cases} M = \text{dec}_K^N(C) & \text{if } \text{vfy}_L^{N,A}(C, T) = \top, \\ \text{abort} & \text{if } \text{vfy}_L^{N,A}(C, T) = \perp, \end{cases} \quad (9b)$$

$$\mathcal{V}_{K,L}^{N,A} : (C, T) \mapsto \top / \perp = \text{vfy}_L^{N,A}(C, T), \quad (9c)$$

where **abort** is a dedicated sign stating that decryption will not proceed.

In the next theorem, we demonstrate that this configuration indeed makes Encrypt-then-MAC PA1 secure.

Theorem 5. *The smart configuration $\Pi_{\text{smart}}^{\text{EtM}}$ for Encrypt-then-MAC is PA1 secure. Stated differently, there exists an extractor $\mathbf{E}$ such that for any adversary $\mathbf{A}$ making q queries to its first oracle and p queries to its second oracle,*

$$\text{refPA1}_{\Pi_{\text{smart}}^{\text{EtM}}}^{\mathbf{E}}(\mathbf{A}) \leq \text{UNF}_{\text{mac}}(\mathbf{B}),$$

for some adversary $\mathbf{B}$ making q queries to its first oracle and p queries to its second oracle.

Proof. Consider a PA1 extractor E that operates as follows on input of any tuple (N, A, C, T) , where we recall that E has access to the query history of $\mathcal{E}_{K,L}$:

- If there is an $M \in \{0, 1\}^*$ and an earlier evaluation $\mathcal{E}_{K,L}(N, A, M) = (C, T)$, return that M .
- Otherwise, return a dedicated sign **abort**.

We have to argue that for any adversary A , it is hard to distinguish $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_{K,L}, \mathcal{D}_{K,L})$ from $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_{K,L}, E)$.

Note that, whenever the adversary makes a repeated query to $\mathcal{O}_2$ or relays a response from $\mathcal{O}_1$ to $\mathcal{O}_2$, the oracle response is identical in both worlds (due to step 1 of E above). Suppose the adversary makes a *fresh* query (N, A, C, T) to $\mathcal{O}_2$. In the ideal world (see step 2 of E above), E outputs **abort**. In the real world (see (9)), $\mathcal{D}_{K,L}$ outputs dedicated sign **abort** *except if* $\text{vfy}_L^{N,A}(C, T) = \top$.

More formally, denote by **BAD** the event that, in the real world $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_{K,L}, \mathcal{D}_{K,L})$, any query (N, A, C, T) that A makes to $\mathcal{D}_{K,L}$ satisfies $\text{vfy}_L^{N,A}(C, T) = \top$. Then, by the fundamental lemma of game playing [33]:

$$\begin{aligned} \text{refPA1}_{\Pi_{\text{smart}}}^E(A) &= |\Pr[A^{\mathcal{E}_K, \mathcal{D}_K} \rightarrow 1] - \Pr[A^{\mathcal{E}_K, E} \rightarrow 1]| \\ &\leq |\Pr[A^{\mathcal{E}_K, \mathcal{D}_K} \rightarrow 1 \mid \neg \text{BAD}] - \Pr[A^{\mathcal{E}_K, E} \rightarrow 1]| \\ &\quad + \Pr[A^{\mathcal{E}_K, \mathcal{D}_K} \text{ sets BAD}] \\ &\leq \Pr[A^{\mathcal{E}_K, \mathcal{D}_K} \text{ sets BAD}] \\ &= \text{UNF}_{\text{mac}}(B), \end{aligned}$$

for some adversary B making q queries to its first oracle and p queries to its second oracle. $\square$ $\square$

4.3.1 Application to Isap v2

Isap v2 [32] is an authenticated encryption scheme that was one of the finalists of the NIST Lightweight Cryptography competition [8]. It was particularly developed with side-channel attacks in mind. Encryption is done using the function **IsapEnc** and message authentication using **IsapMAC**. Both internally call a rekeying function **IsapRK** that generates a subkey from a master key and auxiliary input. In detail, on input of a key K , a nonce N , associated data A , and message M :

- **IsapEnc** first evaluates **IsapRK** on input of key K , nonce N , and an initial value indicating the call comes from encryption, to obtain a subkey K_{enc} . This key is then concatenated with the nonce N and fed through a duplex [34] evaluation to generate a keystream to be added to the message M to obtain ciphertext C . Note that its inverse is identical to **IsapEnc** but with C as input instead of M (as the function encrypts using a keystream).
- **IsapMAC** first evaluates a sponge [35] on input of the nonce N , associated data A , and ciphertext C . Right after absorption of the last block, the outer part of the state, Y , is fed to **IsapRK** together with key K and an initial value indicating the call comes from authentication, to obtain a subkey K_{mac} . This value overwrites Y

in the sponge construction, and a tag is squeezed. (Formally, this is the suffix keyed sponge, or SuKS, construction [36] with IsapRK as the key mixing function.)

The three components of Isap v2, IsapRK , IsapEnc (excluding the IsapRK call), and IsapMAC (excluding the IsapRK call), are all based on a permutation, which is queried for different instantiations, i.e. for different rounds, in each component. In [32], Dobraunig et al. proved that in the random permutation model, where the three components are assumed to be instantiated with three *independent* random permutations π_1 , π_2 , and π_3 , Isap v2 is a leakage resilient authenticated encryption scheme. The proof intuitively consists of first observing that $\text{IsapRK}[\pi_1]$ is a leakage resilient duplex construction [37] and thus behaves like a random oracle RO. Then, $\text{IsapEnc}[\text{RO}, \pi_2]$ (noting that the reliance on $\text{IsapRK}[\pi_1]$ is replaced by RO) is also a leakage resilient duplex construction but operating on different random subkeys per nonce (thanks to RO). Finally, again using random behavior of $\text{IsapRK}[\pi_1]$, $\text{IsapMAC}[\text{RO}, \pi_3]$ becomes the SuKS construction, which is proven to be leakage resilient by Dobraunig and Mennink [36].

Even though the same key is used for encryption and authentication in Isap v2, we can observe that the extractor of Theorem 5 is in fact ignorant of the key usage. This means that we can transform the smart configuration of (9) to Isap v2 verbatim, and PA1 security seems to be achieved. Unfortunately, proving so is not obvious, as, in the proof of Theorem 5, the authentication part is basically “detached” from the authenticated encryption scheme and this can only be done in case of independent key uses. Fortunately, using the fact that $\text{IsapRK}[\pi_1]$ behaves randomly (formalized in the proof below), we achieve a comparable result. We remark that a similar observation necessarily applies to other Encrypt-then-MAC authenticated encryption schemes that derive independent-looking subkeys for encryption and for authentication, such as GCM [6] and Elephant [38, 39].

In detail, consider the following configuration $\Pi_{\text{smart}}^{\text{Isap}} = \{\mathcal{E}, \mathcal{D}, \mathcal{V}\}$ for Isap v2:

$$\mathcal{E}_K^{N,A} : M \mapsto (C, T) = (\text{IsapEnc}[\pi_1, \pi_2]_K^N(M), \text{IsapMAC}[\pi_1, \pi_3]_K^{N,A}(C)), \quad (10a)$$

$$\mathcal{D}_K^{N,A} : (C, T) \mapsto \begin{cases} M = \text{IsapEnc}[\pi_1, \pi_2]_K^N(C) & \text{if } \text{IsapMAC}[\pi_1, \pi_3]_K^{N,A}(C) = T, \\ \text{abort} & \text{if } \text{IsapMAC}[\pi_1, \pi_3]_K^{N,A}(C) \neq T, \end{cases} \quad (10b)$$

$$\mathcal{V}_K^{N,A} : (C, T) \mapsto \top / \perp = \left(\text{IsapMAC}[\pi_1, \pi_3]_K^{N,A}(C) \stackrel{?}{=} T \right). \quad (10c)$$

For this configuration, we obtain the following result.

Theorem 6. *The smart configuration $\Pi_{\text{smart}}^{\text{Isap}}$ for Isap v2 is PA1 secure. Stated differently, let π_1, π_2, π_3 be three random permutations and RO be a random oracle. There exists an extractor E such that for any adversary A making q queries to its first oracle and p queries to its second oracle, and that can make r_i queries to π_i for $i = 1, 2, 3$,*

$$\text{refPA1}_{\Pi_{\text{smart}}^{\text{Isap}}}^E(\mathbf{A}) \leq 2 \cdot \text{PRF}_{\text{IsapRK}[\pi_1]}(\mathbf{B}) + \text{UNF}_{\text{SuKS}[\text{RO}, \pi_3]}(\mathbf{C}), \quad (11)$$

for some adversary $\mathbf{B}$ making $q+p$ queries to its oracle and r_1 queries to the underlying permutation, and some adversary $\mathbf{C}$ making q queries to its first oracle, p queries to its second oracle, and r_3 queries to the underlying permutation.

The proof is given in Supplementary Material A, and we now only give an interpretation of the bound. Observe that bounds for the individual terms in the right hand side of (11) are derived in [32, Theorem 1]. For the PRF-security of IsapRK , we factually have to also take into account the total length in blocks for which $\text{IsapRK}[\pi_1]$ is evaluated. Looking into the algorithm of IsapRK , that length is k blocks, and we obtain [32, Proposition 1]:

$$\text{PRF}_{\text{IsapRK}[\pi_1]}(\mathbf{B}) \leq \frac{8k(q+p)r_1 + 4k^2(q+p)^2}{2^b} + \frac{\binom{4+2k(q+p)+r_1}{2} + \binom{r_1}{2}}{2^b} + \frac{2r_1}{2^k} + \frac{1}{2^k},$$

where b is the width of the underlying permutation. For the UNF-security of SuKS , we rely on its PRF-security and add the incurred tag guessing probability $p/2^t$:

$$\text{UNF}_{\text{SuKS}[\text{RO}, \pi_3]}(\mathbf{C}) \leq \text{PRF}_{\text{SuKS}[\text{RO}, \pi_3]}(\mathbf{D}) + \frac{p}{2^t},$$

for some adversary $\mathbf{D}$ making $q+p$ queries to its oracle and r_3 queries to the underlying permutation. This advantage can subsequently be bounded using [32, Proposition 3] (simplified):

$$\text{UNF}_{\text{SuKS}[\text{RO}, \pi_3]}(\mathbf{C}) \leq \frac{2r_3^2}{2^{2k-1}} + \frac{\mu_{n-k,k}^{2r_3} \cdot r_3}{2^k} + \frac{\mu_{k,n-k}^{2(q+p)} \cdot r_3}{2^{n-k}} + \frac{p}{2^t},$$

where the terms $\mu_{n-k,k}^{2r_3}$ and $\mu_{k,n-k}^{2(q+p)}$ are multicollision terms that are bounded by some constants, i.e., they are in $O(1)$. Together, we obtain the following simplified bound for (11):

$$\begin{aligned} \text{refPA1}_{\Pi_{\text{smart}}^{\text{Isap}}}^{\text{E}}(\mathbf{A}) &\leq \frac{8k(q+p)r_1 + 4k^2(q+p)^2}{2^b} + \frac{\binom{4+2k(q+p)+r_1}{2} + \binom{r_1}{2}}{2^b} \\ &\quad + \frac{2r_1}{2^k} + \frac{1}{2^k} + \frac{2r_3^2}{2^{2k-1}} + \frac{O(1) \cdot r_3}{2^k} + \frac{O(1) \cdot r_3}{2^{b-k}} + \frac{p}{2^t}. \end{aligned}$$

5 Conclusion

Conclusion We gave a refined treatment of plaintext awareness in the context of release of unverified plaintext. The refinement actually matters: Encrypt-then-MAC is (surprisingly) insecure in the original definition but (unsurprisingly) secure in the refined definition. The rationale behind the refined model is surprisingly subtle, and urged us to define the notion of a *valid configuration*. In addition, proving plaintext awareness still remains a nontrivial problem in case the same key is used for encryption and for authentication, as we experienced for Isap v2 in Theorem 6. A more modular reasoning in this case would be desirable, but unlikely to be possible.

Furthermore, as mentioned in Section 2.4, PA2 security implies PA1 security, and the same holds for the refined notions. Consequently, we can conclude from our results of Sections 4.1 that no generic Encrypt-and-MAC construction can be PA2 secure. Given the positive result in Section 4.2 on MAC-then-Encrypt and Section 4.3 on Encrypt-then-MAC, we can be hopeful that for this mode a configuration exists that achieves PA2 security.

SUPPLEMENTARY MATERIAL

Appendix A Proof of Theorem 6

We consider exactly the PA1 extractor E as given in the proof of Theorem 5. We have to argue that for any adversary A , it is hard to distinguish $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_K, \mathcal{D}_K)$ from $(\mathcal{O}_1, \mathcal{O}_2) = (\mathcal{E}_K, E)$. As we now consider the ideal permutation model, A has additional access to the three permutations π_1, π_2, π_3 (in both worlds).

We cannot exactly follow the same reasoning as in the proof of Theorem 5, for the simple reason that the same key K is used in both IsapEnc and IsapMAC . Nevertheless, similar to the proof of [32, Theorem 1], we can rely on the observation that both functions use the key K through $\text{IsapRK}[\pi_1]$, which performs domain separation internally, and which behaves like a random oracle.

In detail, let RO be a random oracle with the same interface as $\text{IsapRK}[\pi_1]$. Recall that our goal is to bound the following advantage of A :

$$\text{refPA1}_{\Pi_{\text{smart}}^{\text{Isap}}}^E(A) = \left| \Pr[A^{\mathcal{E}_K, \mathcal{D}_K, \pi_1, \pi_2, \pi_3} \rightarrow 1] - \Pr[A^{\mathcal{E}_K, E, \pi_1, \pi_2, \pi_3} \rightarrow 1] \right|. \quad (\text{A1})$$

In both the real world and the ideal world, we replace the evaluations of $\text{IsapRK}[\pi_1]$ by a random oracle RO . This results in a renewed definition of $\Pi_{\text{smart}}^{\text{Isap}} = \{\mathcal{E}', \mathcal{D}', \mathcal{V}'\}$ for Isap v2 , where we abuse notation and use $\text{IsapEnc}[RO, \pi_2]$ (resp., $\text{IsapMAC}[RO, \pi_3]$) to denote the encryption scheme (resp., MAC scheme), where $\text{IsapRK}[\pi_1]$ is replaced by RO :

$$\mathcal{E}'_K^{N,A} : M \mapsto (C, T) = (\text{IsapEnc}[RO, \pi_2]_K^N(M), \text{IsapMAC}[RO, \pi_3]_K^{N,A}(C)), \quad (\text{A2a})$$

$$\mathcal{D}'_K^{N,A} : (C, T) \mapsto \begin{cases} M = \text{IsapEnc}[RO, \pi_2]_K^N(C) & \text{if } \text{IsapMAC}[RO, \pi_3]_K^{N,A}(C) = T, \\ \text{abort} & \text{if } \text{IsapMAC}[RO, \pi_3]_K^{N,A}(C) \neq T, \end{cases} \quad (\text{A2b})$$

$$\mathcal{V}'_K^{N,A} : (C, T) \mapsto \top / \perp = \left(\text{IsapMAC}[RO, \pi_3]_K^{N,A}(C) \stackrel{?}{=} T \right). \quad (\text{A2c})$$

We have:

$$(\text{A1}) \leq \left| \Pr[A^{\mathcal{E}'_K, \mathcal{D}'_K, \pi_1, \pi_2, \pi_3} \rightarrow 1] - \Pr[A^{\mathcal{E}'_K, E, \pi_1, \pi_2, \pi_3} \rightarrow 1] \right| + 2 \cdot \text{PRF}_{\text{IsapRK}[\pi_1]}(B), \quad (\text{A3})$$

for some adversary B making $q+p$ queries to its oracle and r_1 queries to the underlying permutation.

For the remaining distance in (A3), note that oracle access to π_1 is meaningless; it can be dropped. A more important observation is that in $\mathcal{E}'_K$ and $\mathcal{D}'_K$, any call to RO through $\text{IsapEnc}[RO, \pi_2]$ is prepended with an initial value indicating the call comes from encryption and any call to RO through $\text{IsapMAC}[RO, \pi_3]$ is prepended with an initial value indicating the call comes from authentication (see the itemized explanation in the beginning of this section). We can basically “detach” the two oracle calls, and conclude that $\text{IsapEnc}[RO, \pi_2]$ and $\text{IsapMAC}[RO, \pi_3]$ share no source of randomness. We can now proceed similar to the proof of Theorem 5. For this, we observe that $\text{IsapMAC}[RO, \pi_3]$ is equivalent to $\text{SuKS}[RO, \pi_3]$ as defined and analyzed by Dobraunig and Mennink [36]. We thus obtain from (A3):

$$(A1) \leq 2 \cdot \text{PRF}_{\text{IsapRK}[\pi_1]}(B) + \text{UNF}_{\text{SuKS}[RO, \pi_3]}(C),$$

for some adversary B as described above and some adversary C making q queries to its first oracle, p queries to its second oracle, and r_3 queries to the underlying permutation.

References

- [1] Katz, J., Yung, M.: Unforgeable encryption and chosen ciphertext secure modes of operation. In: Schneier, B. (ed.) Fast Software Encryption, 7th International Workshop, FSE 2000, New York, NY, USA, April 10-12, 2000, Proceedings. Lecture Notes in Computer Science, vol. 1978, pp. 284–299. Springer, ??? (2000). https://doi.org/10.1007/3-540-44706-7_20 . https://doi.org/10.1007/3-540-44706-7_20
- [2] Bellare, M., Namprempre, C.: Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. In: Okamoto, T. (ed.) Advances in Cryptology - ASIACRYPT 2000, 6th International Conference on the Theory and Application of Cryptology and Information Security, Kyoto, Japan, December 3-7, 2000, Proceedings. Lecture Notes in Computer Science, vol. 1976, pp. 531–545. Springer, ??? (2000). https://doi.org/10.1007/3-540-44448-3_41 . https://doi.org/10.1007/3-540-44448-3_41
- [3] Darmont, J., Petit, B., Schneider, M.: OCB: A generic benchmark to evaluate the performances of object-oriented database systems. In: Schek, H., Saltor, F., Ramos, I., Alonso, G. (eds.) Advances in Database Technology - EDBT’98, 6th International Conference on Extending Database Technology, Valencia, Spain, March 23-27, 1998, Proceedings. Lecture Notes in Computer Science, vol. 1377, pp. 326–340. Springer, ??? (1998). <https://doi.org/10.1007/BFb0100994> . <https://doi.org/10.1007/BFb0100994>
- [4] Rogaway, P., Bellare, M., Black, J., Krovetz, T.: OCB: a block-cipher mode of operation for efficient authenticated encryption. In: Reiter, M.K., Samarati, P. (eds.) CCS 2001, Proceedings of the 8th ACM Conference on Computer and

- Communications Security, Philadelphia, Pennsylvania, USA, November 6-8, 2001, pp. 196–205. ACM, ??? (2001). <https://doi.org/10.1145/501983.502011> . <https://doi.org/10.1145/501983.502011>
- [5] Rogaway, P., Bellare, M., Black, J.: OCB: A block-cipher mode of operation for efficient authenticated encryption. *ACM Trans. Inf. Syst. Secur.* **6**(3), 365–403 (2003) <https://doi.org/10.1145/937527.937529>
 - [6] McGrew, D.A., Viega, J.: The security and performance of the galois/counter mode (GCM) of operation. In: Canteaut, A., Viswanathan, K. (eds.) *Progress in Cryptology - INDOCRYPT 2004*, 5th International Conference on Cryptology in India, Chennai, India, December 20-22, 2004, Proceedings. *Lecture Notes in Computer Science*, vol. 3348, pp. 343–355. Springer, ??? (2004). https://doi.org/10.1007/978-3-540-30556-9_27 . https://doi.org/10.1007/978-3-540-30556-9_27
 - [7] CAESAR: Competition for Authenticated Encryption: Security, Applicability, and Robustness. <http://competitions.cr.yp.to/caesar.html>
 - [8] NIST Lightweight Cryptography. <https://csrc.nist.gov/Projects/lightweight-cryptography>
 - [9] Hoang, V.T., Reyhanitabar, R., Rogaway, P., Vizár, D.: Online authenticated-encryption and its nonce-reuse misuse-resistance. In: Gennaro, R., Robshaw, M. (eds.) *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference*, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 9215, pp. 493–517. Springer, ??? (2015). https://doi.org/10.1007/978-3-662-47989-6_24 . https://doi.org/10.1007/978-3-662-47989-6_24
 - [10] Berti, F., Koeune, F., Pereira, O., Peters, T., Standaert, F.: Leakage-resilient and misuse-resistant authenticated encryption. *IACR Cryptol. ePrint Arch.*, 996 (2016)
 - [11] Barwell, G., Martin, D.P., Oswald, E., Stam, M.: Authenticated encryption in the face of protocol and side channel leakage. In: Takagi, T., Peyrin, T. (eds.) *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security*, Hong Kong, China, December 3-7, 2017, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 10624, pp. 693–723. Springer, ??? (2017). https://doi.org/10.1007/978-3-319-70694-8_24 . https://doi.org/10.1007/978-3-319-70694-8_24
 - [12] Andreeva, E., Bogdanov, A., Luykx, A., Mennink, B., Mouha, N., Yasuda, K.: How to securely release unverified plaintext in authenticated encryption. In: Sarkar, P., Iwata, T. (eds.) *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security*, Kaoshiung, Taiwan, R.O.C., December 7-11, 2014. Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 8873, pp. 105–125. Springer, ???

- (2014). https://doi.org/10.1007/978-3-662-45611-8_6 . https://doi.org/10.1007/978-3-662-45611-8_6
- [13] Bellare, M., Rogaway, P.: Optimal asymmetric encryption. In: Santis, A.D. (ed.) *Advances in Cryptology - EUROCRYPT '94, Workshop on the Theory and Application of Cryptographic Techniques*, Perugia, Italy, May 9-12, 1994, Proceedings. *Lecture Notes in Computer Science*, vol. 950, pp. 92–111. Springer, ??? (1994). <https://doi.org/10.1007/BFb0053428> . <https://doi.org/10.1007/BFb0053428>
 - [14] Bellare, M., Palacio, A.: Towards plaintext-aware public-key encryption without random oracles. In: Lee, P.J. (ed.) *Advances in Cryptology - ASIACRYPT 2004, 10th International Conference on the Theory and Application of Cryptology and Information Security*, Jeju Island, Korea, December 5-9, 2004, Proceedings. *Lecture Notes in Computer Science*, vol. 3329, pp. 48–62. Springer, ??? (2004). https://doi.org/10.1007/978-3-540-30539-2_4 . https://doi.org/10.1007/978-3-540-30539-2_4
 - [15] Datta, N., Luykx, A., Mennink, B., Nandi, M.: Understanding RUP integrity of COLM. *IACR Trans. Symmetric Cryptol.* **2017**(2), 143–161 (2017) <https://doi.org/10.13154/tosc.v2017.i2.143-161>
 - [16] Chakraborti, A., Datta, N., Nandi, M.: INT-RUP analysis of block-cipher based authenticated encryption schemes. In: Sako, K. (ed.) *Topics in Cryptology - CT-RSA 2016 - The Cryptographers' Track at the RSA Conference 2016*, San Francisco, CA, USA, February 29 - March 4, 2016, Proceedings. *Lecture Notes in Computer Science*, vol. 9610, pp. 39–54. Springer, ??? (2016). https://doi.org/10.1007/978-3-319-29485-8_3 . https://doi.org/10.1007/978-3-319-29485-8_3
 - [17] Imamura, K., Minematsu, K., Iwata, T.: Integrity analysis of authenticated encryption based on stream ciphers. *Int. J. Inf. Sec.* **17**(5), 493–511 (2018) <https://doi.org/10.1007/s10207-017-0378-9>
 - [18] Zhang, P., Wang, P., Hu, H., Cheng, C., Kuai, W.: INT-RUP security of checksum-based authenticated encryption. In: Okamoto, T., Yu, Y., Au, M.H., Li, Y. (eds.) *Provable Security - 11th International Conference, ProvSec 2017, Xi'an, China, October 23-25, 2017, Proceedings*. *Lecture Notes in Computer Science*, vol. 10592, pp. 147–166. Springer, ??? (2017). https://doi.org/10.1007/978-3-319-68637-0_9 . https://doi.org/10.1007/978-3-319-68637-0_9
 - [19] Hirose, S., Sasaki, Y., Yasuda, K.: Rate-one AE with security under RUP. In: Nguyen, P.Q., Zhou, J. (eds.) *Information Security - 20th International Conference, ISC 2017, Ho Chi Minh City, Vietnam, November 22-24, 2017, Proceedings*. *Lecture Notes in Computer Science*, vol. 10599, pp. 3–20. Springer, ??? (2017). https://doi.org/10.1007/978-3-319-69659-1_1 . https://doi.org/10.1007/978-3-319-69659-1_1

- [20] Ashur, T., Dunkelman, O., Luykx, A.: Boosting authenticated encryption robustness with minimal modifications. In: Katz, J., Shacham, H. (eds.) *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part III. *Lecture Notes in Computer Science*, vol. 10403, pp. 3–33. Springer, ??? (2017). https://doi.org/10.1007/978-3-319-63697-9_1 . https://doi.org/10.1007/978-3-319-63697-9_1
- [21] Chang, D., Datta, N., Dutta, A., Mennink, B., Nandi, M., Sanadhya, S., Sibleyras, F.: Release of unverified plaintext: Tight unified model and application to ANYDAE. *IACR Trans. Symmetric Cryptol.* **2019**(4), 119–146 (2019) <https://doi.org/10.13154/tosc.v2019.i4.119-146>
- [22] Fleischmann, E., Forler, C., Lucks, S.: Mcoe: A family of almost foolproof on-line authenticated encryption schemes. In: Canteaut, A. (ed.) *Fast Software Encryption - 19th International Workshop, FSE 2012, Washington, DC, USA, March 19-21, 2012. Revised Selected Papers. Lecture Notes in Computer Science*, vol. 7549, pp. 196–215. Springer, ??? (2012). https://doi.org/10.1007/978-3-642-34047-5_12 . https://doi.org/10.1007/978-3-642-34047-5_12
- [23] Andreeva, E., Bilgin, B., Bogdanov, A., Luykx, A., Mennink, B., Mouha, N., Yasuda, K.: APE: authenticated permutation-based encryption for lightweight cryptography. In: Cid, C., Rechberger, C. (eds.) *Fast Software Encryption - 21st International Workshop, FSE 2014, London, UK, March 3-5, 2014. Revised Selected Papers. Lecture Notes in Computer Science*, vol. 8540, pp. 168–186. Springer, ??? (2014). https://doi.org/10.1007/978-3-662-46706-0_9 . https://doi.org/10.1007/978-3-662-46706-0_9
- [24] Iwata, T., Khairallah, M., Minematsu, K., Peyrin, T.: New Results on Romulus. In: *NIST Lightweight Cryptography Workshop (2020 (2006))*. <https://csrc.nist.gov/CSRC/media/Events/lightweight-cryptography-workshop-2020/documents/papers/new-results-romulus-lwc2020.pdf>
- [25] Guo, C., Iwata, T., Khairallah, M., Minematsu, K., Peyrin, T.: Romulus v1.3. Submission To The NIST Lightweight Cryptography Standardization Process (2021). <https://romulusae.github.io/romulus/>
- [26] Boldyreva, A., Degabriele, J.P., Paterson, K.G., Stam, M.: On symmetric encryption with distinguishable decryption failures. In: Moriai, S. (ed.) *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11-13, 2013. Revised Selected Papers. Lecture Notes in Computer Science*, vol. 8424, pp. 367–390. Springer, ??? (2013). https://doi.org/10.1007/978-3-662-43933-3_19 . https://doi.org/10.1007/978-3-662-43933-3_19
- [27] Hoang, V.T., Krovetz, T., Rogaway, P.: Robust authenticated-encryption AEZ and the problem that it solves. In: Oswald, E., Fischlin, M. (eds.) *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the*

- Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26–30, 2015, Proceedings, Part I. Lecture Notes in Computer Science, vol. 9056, pp. 15–44. Springer, ??? (2015). https://doi.org/10.1007/978-3-662-46800-5_2 . https://doi.org/10.1007/978-3-662-46800-5_2
- [28] Barwell, G., Page, D., Stam, M.: Rogue decryption failures: Reconciling AE robustness notions. In: Groth, J. (ed.) *Cryptography and Coding - 15th IMA International Conference, IMACC 2015*, Oxford, UK, December 15–17, 2015. Proceedings. Lecture Notes in Computer Science, vol. 9496, pp. 94–111. Springer, ??? (2015). https://doi.org/10.1007/978-3-319-27239-9_6 . https://doi.org/10.1007/978-3-319-27239-9_6
 - [29] Degabriele, J.P., Fischlin, M.: Simulatable channels: Extended security that is universally composable and easier to prove. In: Peyrin, T., Galbraith, S.D. (eds.) *Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security*, Brisbane, QLD, Australia, December 2–6, 2018, Proceedings, Part III. Lecture Notes in Computer Science, vol. 11274, pp. 519–550. Springer, ??? (2018). https://doi.org/10.1007/978-3-030-03332-3_19 . https://doi.org/10.1007/978-3-030-03332-3_19
 - [30] Rogaway, P., Shrimpton, T.: A provable-security treatment of the key-wrap problem. In: Vaudenay, S. (ed.) *Advances in Cryptology - EUROCRYPT 2006*, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006, Proceedings. Lecture Notes in Computer Science, vol. 4004, pp. 373–390. Springer, ??? (2006). https://doi.org/10.1007/11761679_23 . https://doi.org/10.1007/11761679_23
 - [31] Bellare, M., Rogaway, P., Wagner, D.A.: The EAX mode of operation. In: Roy, B.K., Meier, W. (eds.) *Fast Software Encryption*, 11th International Workshop, FSE 2004, Delhi, India, February 5–7, 2004, Revised Papers. Lecture Notes in Computer Science, vol. 3017, pp. 389–407. Springer, ??? (2004). https://doi.org/10.1007/978-3-540-25937-4_25 . https://doi.org/10.1007/978-3-540-25937-4_25
 - [32] Dobraunig, C., Eichlseder, M., Mangard, S., Mendel, F., Mennink, B., Primas, R., Unterluggauer, T.: Isap v2.0. *IACR Transactions on Symmetric Cryptology* **2020**(S1), 390–416 (2020) <https://doi.org/10.13154/tosc.v2020.iS1.390-416>
 - [33] Bellare, M., Rogaway, P.: The security of triple encryption and a framework for code-based game-playing proofs. In: Vaudenay, S. (ed.) *Advances in Cryptology - EUROCRYPT 2006*, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006, Proceedings. Lecture Notes in Computer Science, vol. 4004, pp. 409–426. Springer, ??? (2006). https://doi.org/10.1007/11761679_25 . https://doi.org/10.1007/11761679_25
 - [34] Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Duplexing the sponge:

- Single-pass authenticated encryption and other applications. In: Miri, A., Vaudenay, S. (eds.) *Selected Areas in Cryptography - 18th International Workshop, SAC 2011, Toronto, ON, Canada, August 11-12, 2011, Revised Selected Papers*. Lecture Notes in Computer Science, vol. 7118, pp. 320–337. Springer, ??? (2011). https://doi.org/10.1007/978-3-642-28496-0_19 . https://doi.org/10.1007/978-3-642-28496-0_19
- [35] Bertoni, G.M., Daemen, J., Peeters, M., Van Assche, G.: Sponge functions. *ECRYPT Hash Workshop 2007* (2007)
- [36] Dobraunig, C., Mennink, B.: Security of the suffix keyed sponge. *IACR Trans. Symmetric Cryptol.* **2019**(4), 223–248 (2019) <https://doi.org/10.13154/tosc.v2019.i4.223-248>
- [37] Dobraunig, C., Mennink, B.: Leakage resilience of the duplex construction. In: Galbraith, S.D., Moriai, S. (eds.) *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8-12, 2019, Proceedings, Part III*. Lecture Notes in Computer Science, vol. 11923, pp. 225–255. Springer, ??? (2019). https://doi.org/10.1007/978-3-030-34618-8_8 . https://doi.org/10.1007/978-3-030-34618-8_8
- [38] Beyne, T., Chen, Y.L., Dobraunig, C., Mennink, B.: Elephant v2 (2021). <https://csrc.nist.gov/CSRC/media/Projects/lightweight-cryptography/documents/finalist-round/updated-spec-doc/elephant-spec-final.pdf>
- [39] Beyne, T., Chen, Y.L., Dobraunig, C., Mennink, B.: Dumbo, Jumbo, and Delirium: Parallel authenticated encryption for the lightweight circus. *IACR Trans. Symmetric Cryptol.* **2020**(S1), 5–30 (2020) <https://doi.org/10.13154/tosc.v2020.iS1.5-30>